

ADR-003: Elección de Patrón de Aplicación – CQRS vs CRUD Tradicional

Estado

Aceptado

Fecha

19/06/2026

Decisión

Adoptar el patrón CQRS (Command Query Responsibility Segregation) para separar las operaciones de lectura y escritura de datos dentro de la aplicación, utilizando modelos de persistencia independientes para cada tipo de operación.

1. Contexto

El proyecto requiere exponer servicios para la gestión y consulta de información médica mediante APIs REST desarrolladas en Java Spring Boot WebFlux, desplegadas sobre una arquitectura de microservicios en Azure Cloud.

Los requerimientos incluyen:

- Consultas con criterios específicos de búsqueda.
- Diferentes modelos de respuesta según el caso de uso.
- Separación clara entre operaciones de lectura y escritura.
- Escalabilidad independiente para consultas y operaciones transaccionales.
- Facilidad de mantenimiento y evolución de los servicios.
- Integración basada en eventos entre microservicios.
- Compatibilidad con una Arquitectura Hexagonal basada en puertos y adaptadores.
- Optimización de consultas para distintos perfiles de usuario (pacientes, médicos y personal administrativo).

Se busca una solución que permita mantener una estructura organizada del código, optimizar el acceso a los datos y facilitar la evolución futura del sistema.

2. Decisión

Se adopta el patrón CQRS (Command Query Responsibility Segregation) como modelo de organización de los casos de uso de la aplicación.

Las operaciones de escritura (Commands) serán implementadas mediante servicios especializados que persistirán la información transaccional en Azure SQL Database, considerado como la fuente principal de verdad del sistema.

Las operaciones de lectura (Queries) serán implementadas mediante servicios independientes que accederán a modelos de consulta optimizados almacenados en Azure Cosmos DB.

La sincronización entre los modelos de escritura y lectura se realizará mediante eventos de integración publicados y consumidos a través de Azure Service Bus.

No se implementará Event Sourcing. Los eventos serán utilizados únicamente para la propagación de cambios y actualización de los modelos de lectura, manteniendo Azure SQL Database como fuente oficial de persistencia.

3. Alternativas Consideradas

A. CQRS (Seleccionada)

Pros

- Separación clara entre consultas y modificaciones de datos.
- Mayor mantenibilidad del código.
- Facilita la aplicación del principio de responsabilidad única (SRP).
- Permite optimizar consultas sin afectar la lógica de negocio transaccional.
- Mejora la organización de casos de uso complejos.
- Facilita la escalabilidad independiente de lectura y escritura.
- Se integra naturalmente con Arquitectura Hexagonal.
- Facilita futuras evoluciones hacia arquitecturas orientadas a eventos.
- Permite utilizar tecnologías especializadas para cada necesidad de persistencia.
- Reduce la carga sobre los sistemas transaccionales mediante modelos de lectura optimizados.

Contras

- Incrementa la cantidad de clases y componentes.
- Requiere disciplina en la separación de responsabilidades.
- Mayor complejidad respecto a un enfoque CRUD tradicional.
- Introduce sincronización entre modelos de lectura y escritura.
- Requiere monitoreo de eventos y proyecciones.
- Puede existir consistencia eventual entre los modelos de datos.

B. CRUD Tradicional

Pros

- Simplicidad de implementación.
- Menor cantidad de componentes.
- Curva de aprendizaje reducida.
- Adecuado para aplicaciones pequeñas o con reglas simples.
- Menor complejidad operativa.

Contras

- Mezcla operaciones de lectura y escritura en los mismos servicios.
- Mayor acoplamiento entre responsabilidades.
- Dificulta la evolución de consultas complejas.
- Reduce la claridad de los casos de uso del negocio.
- Puede generar servicios con demasiadas responsabilidades.
- Limita la optimización independiente de consultas y transacciones.
- Menor alineamiento con arquitecturas distribuidas y de microservicios.

4. Consecuencias

Positivas

- Mejor organización del código fuente.
- Casos de uso más fáciles de entender y mantener.
- Separación explícita entre lógica de consulta y lógica transaccional.
- Mayor facilidad para incorporar nuevos requerimientos de lectura.
- Mejor alineamiento con Arquitectura Hexagonal y principios SOLID.
- Escalabilidad independiente para operaciones de lectura y escritura.
- Optimización de consultas mediante modelos especializados en Cosmos DB.
- Reducción de carga sobre las bases de datos transaccionales.
- Mayor desacoplamiento entre componentes mediante eventos.
- Mejor preparación para futuras necesidades de crecimiento del sistema.

Negativas

- Incremento moderado de la cantidad de clases.
- Necesidad de capacitación inicial del equipo.
- Mayor esfuerzo inicial de diseño respecto a una implementación CRUD tradicional.
- Complejidad adicional en la sincronización de datos.
- Posibilidad de consistencia eventual entre Azure SQL Database y Azure Cosmos DB.
- Mayor complejidad de monitoreo y observabilidad.
- Incremento de costos de infraestructura por el uso de múltiples tecnologías de persistencia.

5. Estado Final

Se aprueba el uso de CQRS como patrón de organización de los casos de uso de la aplicación.

Los nuevos desarrollos deberán implementar de forma separada los comandos (Commands) y consultas (Queries), manteniendo responsabilidades independientes para cada tipo de operación.

Las operaciones transaccionales utilizarán Azure SQL Database como almacenamiento principal, mientras que las consultas utilizarán modelos optimizados almacenados en Azure Cosmos DB.

La sincronización entre ambos modelos se realizará mediante eventos de integración gestionados a través de Azure Service Bus.